在 AI 平台筆記本上的原型設計模型

來源：https://codelabs.developers.google.com/codelabs/prototyping-caip-notebooks?hl=zh-tw

步驟數：9

在本研究室中，您將學習如何使用 AI 平台筆記本建立機器學習工作流程原型。我們將說明如何建立自訂筆記本執行個體、在 Git 中追蹤筆記本程式碼，以及使用 What-If Tool 對模型進行偵錯。

目錄

1. 1. 總覽
2. 2. 建立 AI 平台 Notebooks 執行個體
3. 3. 將 BigQuery 資料連結至筆記本
4. 4. 初始化 Git
5. 5. 建構及訓練 TensorFlow 模型
6. 6. 直接從筆記本使用 What-If Tool
7. 7. 選用：將本機 Git 存放區連結至 GitHub
8. 8. 恭喜！
9. 9. 清除所用資源

步驟 1 · 約 1 分鐘

1. 總覽

本實驗室將逐步說明 AI Platform Notebooks 中的各種工具，協助您探索資料及製作機器學習模型原型。

課程內容

內容如下：

- 建立及自訂 AI Platform Notebooks 執行個體
- 透過直接整合至 AI 平台筆記本的 Git，追蹤筆記本程式碼
- 在筆記本中使用 What-If Tool

在 Google Cloud 上執行這個實驗室的總費用約為 **$1 美元**。如需 AI Platform Notebooks 的定價詳細資料，請參閱這裡。

步驟 2 · 約 10 分鐘

2. 建立 AI 平台 Notebooks 執行個體

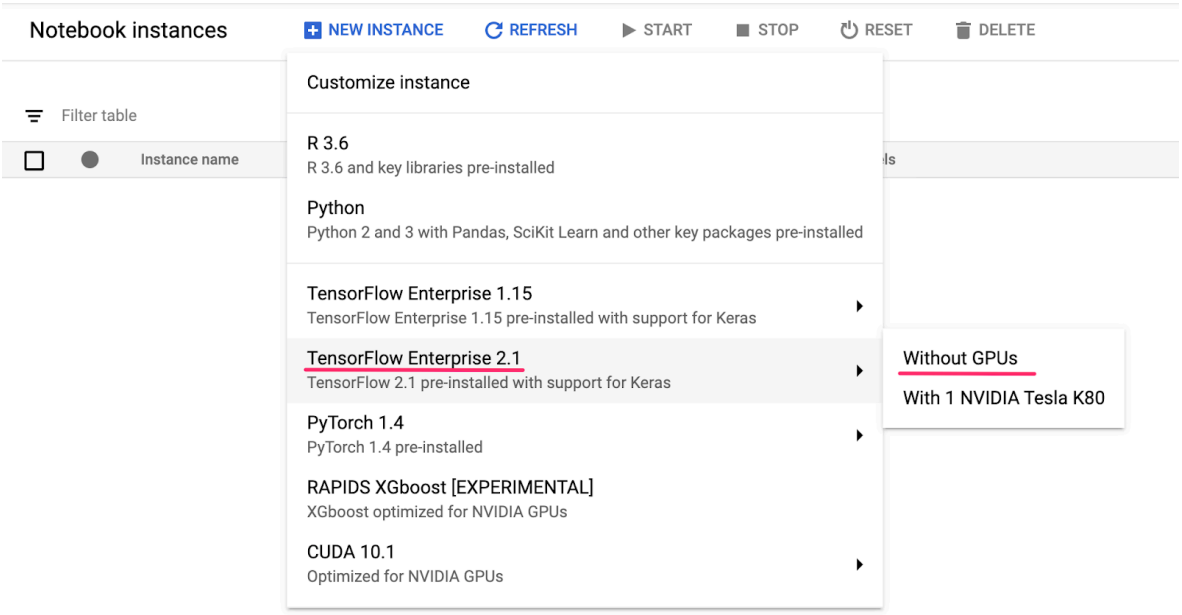
您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照這裡的操作說明進行。

步驟 2：啟用 Compute Engine API

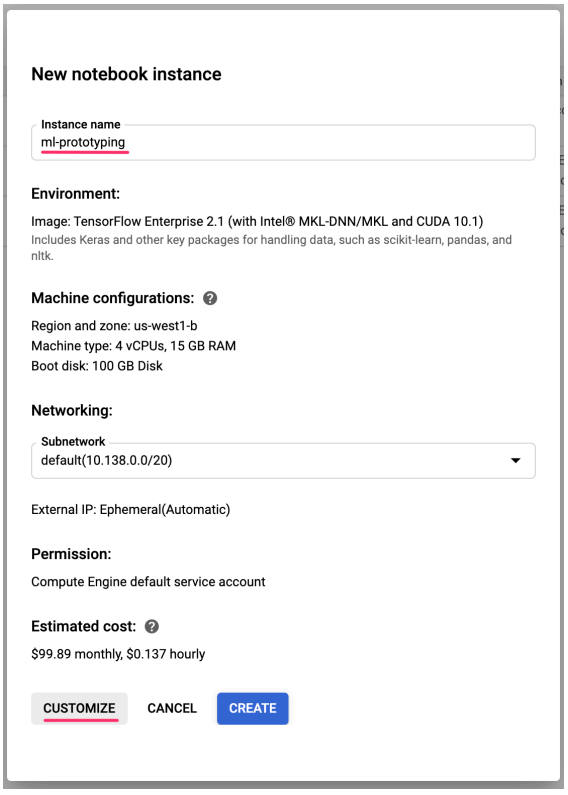
前往「Compute Engine」，然後選取「啟用」 (如果尚未啟用)。您需要這項資訊才能建立筆記本執行個體。

步驟 3：建立筆記本執行個體

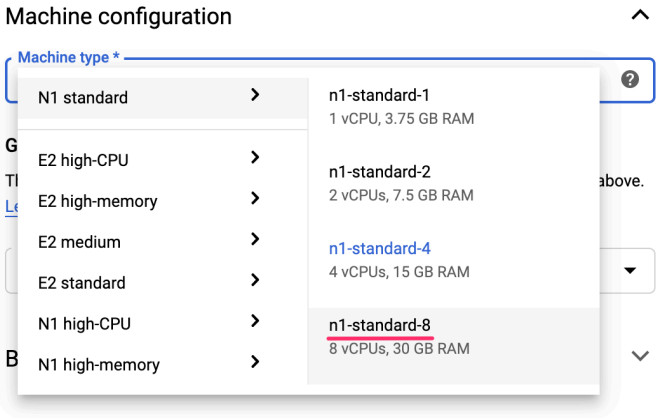
前往 Cloud Console 的 AI Platform Notebooks 專區，然後點選「建立執行個體」。然後選取最新版的「TensorFlow 2 Enterprise」執行個體類型，且「不加入任何 GPU」：



為執行個體命名或使用預設名稱。接著會探討自訂選項。按一下「自訂」按鈕：

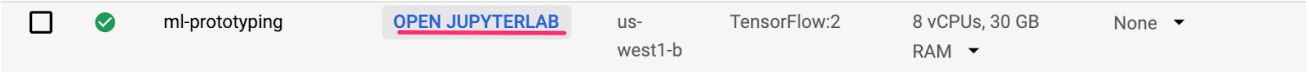


AI Platform Notebooks 提供許多不同的自訂選項，包括執行個體部署的區域、映像檔類型、機器大小、GPU 數量等。我們會使用區域和環境的預設值。機器設定方面，我們將使用 **n1-standard-8** 機器：

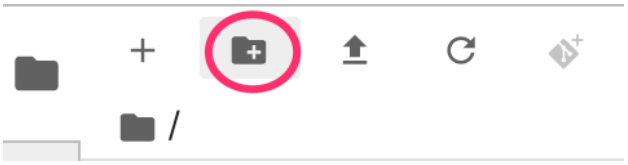


我們不會新增任何 GPU，且會使用開機磁碟、網路和權限的預設值。選取「Create」(建立) 即可建立執行個體。這項作業需要幾分鐘才能完成。

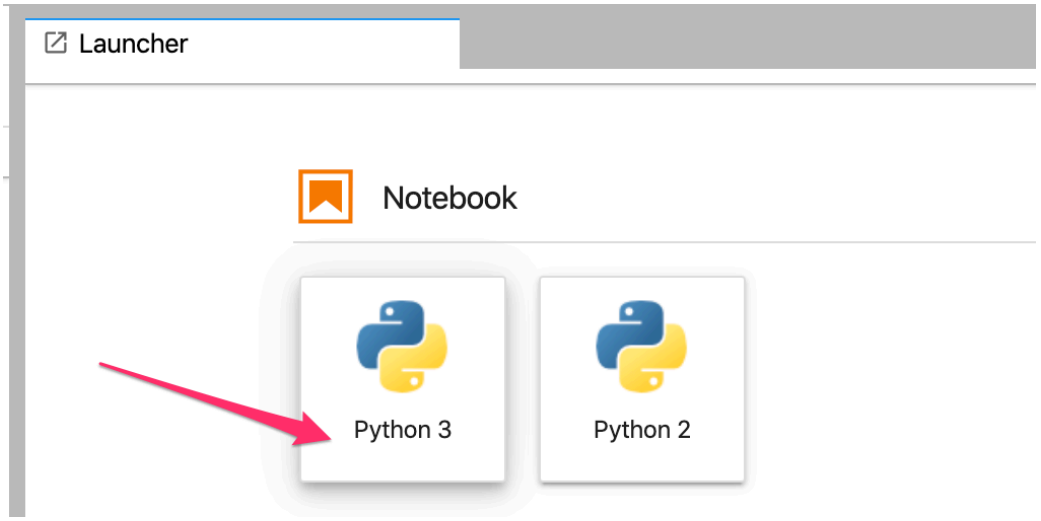
建立執行個體後，Notebooks 使用者介面中的執行個體旁會顯示綠色勾號。選取「Open JupyterLab」開啟執行個體，然後開始製作原型：



開啟執行個體後，建立名為「codelab」的新目錄。在本實驗室中，我們將使用這個目錄：



按兩下新建立的 **codelab** 目錄，然後從啟動器選取 Python 3 筆記本：



將筆記本重新命名為 `demo.ipynb`，或您想使用的任何名稱。

步驟 4：匯入 Python 套件

在筆記本中建立新儲存格，並匯入本程式碼研究室中使用的程式庫：

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

import numpy as np
import json

from sklearn.model_selection import train_test_split
from sklearn.utils import shuffle
from google.cloud import bigquery
from witwidget.notebook.visualization import WitWidget, WitConfigBuilder
```

步驟 3 · 約 5 分鐘

3. 將 BigQuery 資料連結至筆記本

BigQuery 是 Google Cloud 的大數據倉儲，已公開提供許多資料集，供您探索。AI 平台 Notebooks 支援直接整合 BigQuery，不需要驗證。

在本實驗室中，我們將使用出生率資料集。這項資料包含美國 40 年時間範圍內幾乎所有出生記錄的資料，包括嬰兒的出生體重，以及嬰兒父母的人口統計資料。我們會使用部分特徵來預測嬰兒的出生體重。

步驟 1：將 BigQuery 資料下載至筆記本

我們會使用 BigQuery 的 Python 用戶端程式庫，將資料下載至 Pandas DataFrame。原始資料集為 21 GB，包含 1.23 億列。為簡化流程，我們只會使用資料集中的 10,000 個資料列。

使用下列程式碼建構查詢，並預覽產生的 DataFrame。這裡我們從原始資料集取得 4 項特徵，以及嬰兒體重 (模型將預測的項目)。資料集可追溯至多年前，但我們只會使用 2000 年後的資料：

```
query="""
SELECT
  weight_pounds,
  is_male,
  mother_age,
  plurality,
  gestation_weeks
FROM
  publicdata.samples.natality
WHERE year > 2000
LIMIT 10000
"""
df = bigquery.Client().query(query).to_dataframe()
df.head()
```

如要取得資料集中數值特徵的摘要，請執行下列指令：

```
df.describe()
```

這會顯示數值資料欄的平均值、標準差、最小值和其他指標。最後，我們來取得布林值資料欄的資料，指出嬰兒的性別。我們可以使用 Pandas `value_counts` 方法執行這項操作：

```
df['is_male'].value_counts()
```

資料集似乎已根據性別平均分配 (50/50)。

> 如果是較大的資料集，請使用 BigQuery 連接器，而非 Pandas 整合。

步驟 2：準備訓練資料集

我們已將資料集下載至筆記本，做為 Pandas DataFrame，現在可以進行一些前置處理，並將資料集分割為訓練集和測試集。

首先，從資料集中捨棄含有空值的資料列，然後隨機排序資料：

```
df = df.dropna()
df = shuffle(df, random_state=2)
```

接著，將標籤欄位擷取至個別變數，並建立只含特徵的 DataFrame。由於 `is_male` 是布林值，我們會將其轉換為整數，讓模型的所有輸入值都是數值：

```
labels = df['weight_pounds']
data = df.drop(columns=['weight_pounds'])
data['is_male'] = data['is_male'].astype(int)
```

現在，如果您執行 `data.head()` 預覽資料集，應該會看到我們將用於訓練的四項特徵。

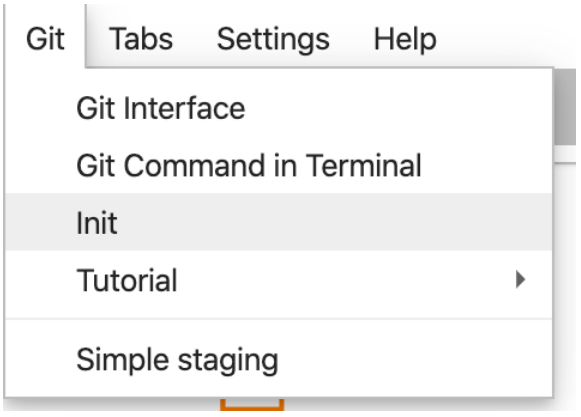
步驟 4 · 約 5 分鐘

4. 初始化 Git

AI Platform Notebooks 直接整合了 Git，因此您可以在筆記本環境中直接進行版本管控。這項功能支援直接在筆記本 UI 中提交程式碼，或透過 JupyterLab 提供的終端機提交。在本節中，我們會在筆記本中初始化 Git 存放區，並透過 UI 進行第一次提交。

步驟 1：初始化 Git 存放區

在 Codelab 目錄中，依序選取「Git」和「Init」 (位於 JupyterLab 頂端選單列)：

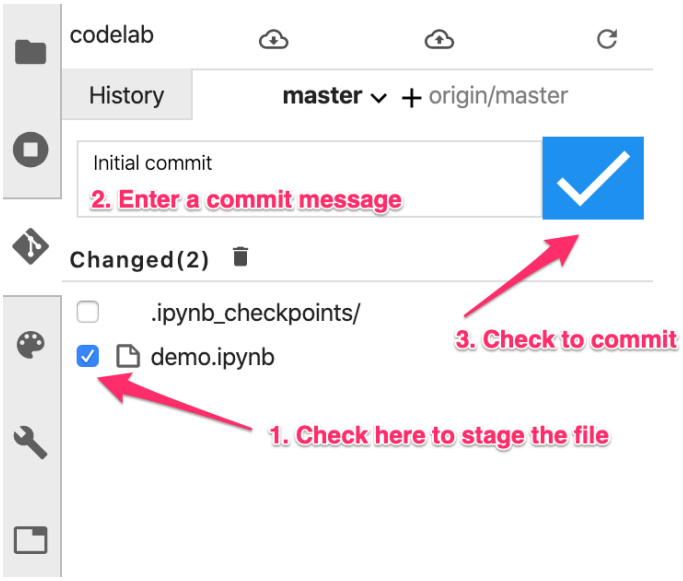


系統詢問是否要將這個目錄設為 Git Repo 時，請選取「Yes」。接著選取左側邊欄的 Git 圖示，即可查看檔案和提交的狀態：

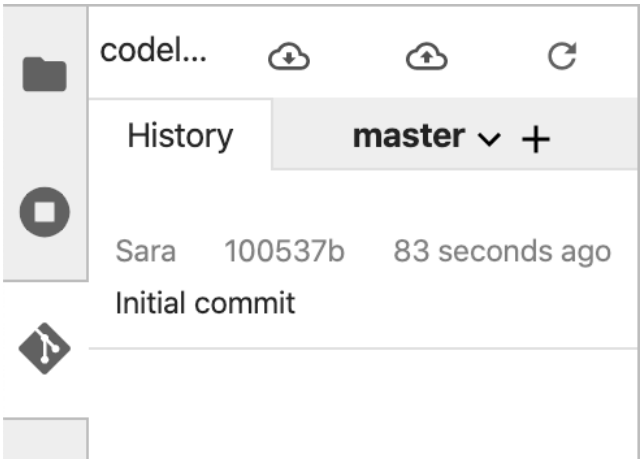
步驟 2：進行首次提交

在這個 UI 中，您可以將檔案新增至修訂版本、查看檔案差異 (稍後會說明)，以及提交變更。首先，請提交剛才新增的筆記本檔案。

勾選 `demo.ipynb` 筆記本檔案旁邊的方塊，將檔案暫存以供修訂 (你可以忽略 `.ipynb_checkpoints/` 目錄)。在文字方塊中輸入提交訊息，然後按一下勾號，提交變更：



系統提示時，請輸入您的姓名和電子郵件地址。然後返回「History」(記錄) 分頁，查看第一個提交內容：



請注意，由於本實驗室發布後經過更新，螢幕截圖可能與您的 UI 不完全相符。

步驟 5 · 約 10 分鐘

5. 建構及訓練 TensorFlow 模型

我們會使用下載到筆記本的 BigQuery 出生率資料集，建立預測嬰兒體重的模型。本實驗室的重點是筆記本工具，而非模型本身的準確度。

步驟 1：將資料分成訓練集和測試集

我們會使用 Scikit Learn `train_test_split` 公用程式分割資料，再建構模型：

```
x,y = data,labels
x_train,x_test,y_train,y_test = train_test_split(x,y)
```

現在，我們準備建構 TensorFlow 模型！

步驟 2：建構及訓練 TensorFlow 模型

我們會使用 tf.keras `Sequential` 模型 API 建構這個模型，將模型定義為一疊層。建構模型所需的所有程式碼如下：

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(len(x_train.iloc[0]),)),
    Dense(32, activation='relu'),
    Dense(1)]
)
```

接著編譯模型，以便訓練。在這裡，我們會選擇模型的最佳化器、損失函式和指標，模型會在訓練期間記錄這些指標。由於這是迴歸模型 (預測數值)，因此我們使用均方誤差而非準確率做為指標：

```
model.compile(optimizer=tf.keras.optimizers.RMSprop(),
              loss=tf.keras.losses.MeanSquaredError(),
              metrics=['mae', 'mse'])
```

您可以使用 Keras 的實用 `model.summary()` 函式，查看模型在每個層級的可訓練參數形狀和數量。

現在可以開始訓練模型了。我們只需要呼叫 `fit()` 方法，並傳遞訓練資料和標籤。我們將使用選用的 validation_split 參數，保留部分訓練資料，以便在每個步驟驗證模型。理想情況下，您會希望訓練和驗證損失*雙雙*減少。但請注意，在這個範例中，我們更著重於模型和筆記本工具，而非模型品質：

```
model.fit(x_train, y_train, epochs=10, validation_split=0.1)
```

步驟 3：針對測試範例生成預測

如要瞭解模型的效能，請針對測試資料集的前 10 個範例產生一些測試預測。

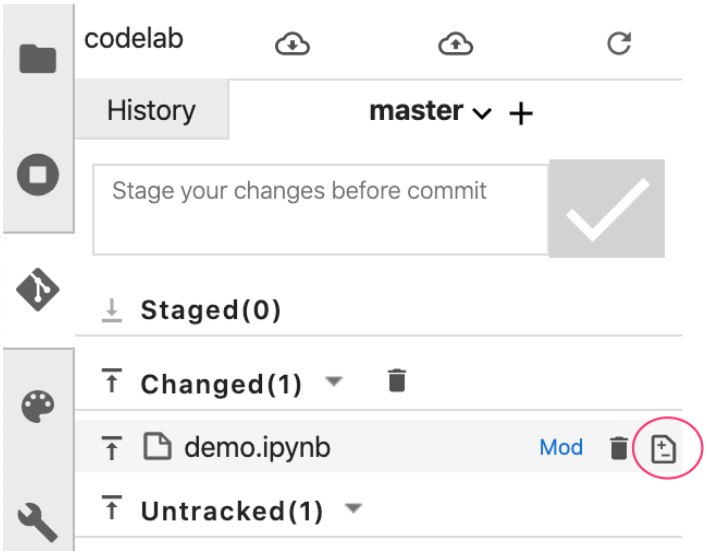
```
num_examples = 10
predictions = model.predict(x_test[:num_examples])
```

接著，我們會疊代模型的預測結果，並與實際值進行比較：

```
for i in range(num_examples):
    print('Predicted val: ', predictions[i][0])
    print('Actual val: ',y_test.iloc[i])
    print()
```

步驟 4：使用 git diff 並提交變更

您已對筆記本進行一些變更，現在可以試用 Notebooks Git UI 中的 Git 差異功能。`demo.ipynb` 筆記本現在應位於使用者介面的「已變更」部分。將滑鼠游標懸停在檔案名稱上，然後按一下差異圖示：



這樣一來，您應該就能看到變更的差異，如下所示：

這次我們將使用終端機，透過指令列提交變更。在 JupyterLab 頂端選單列的「Git」選單中，選取「Git Command in Terminal」。如果您在執行下列指令時開啟左側邊欄的 Git 分頁，就能在 Git UI 中看到變更。

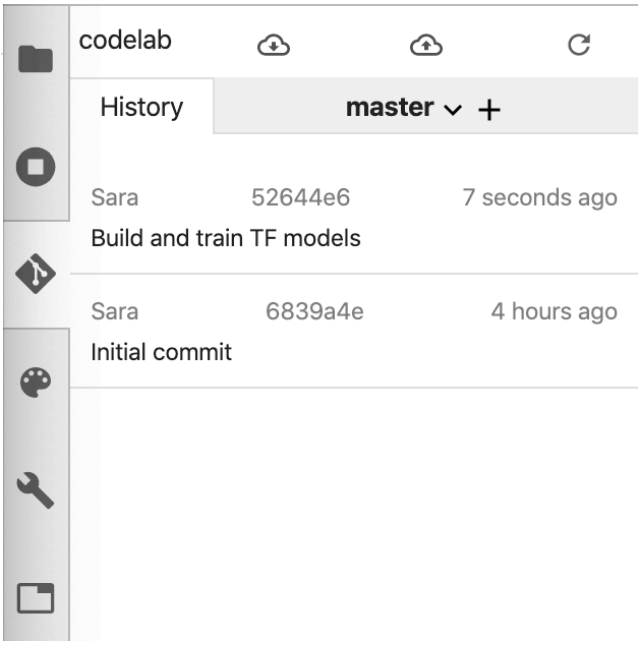
在新的終端機執行個體中，執行下列指令，將筆記本檔案暫存以供提交：

```
git add demo.ipynb
```

然後執行下列指令來提交變更 (您可以使用任何想要的提交訊息)：

```
git commit -m "Build and train TF model"
```

接著，您應該會在記錄中看到最新修訂版本：



步驟 6 · 約 10 分鐘

6. 直接從筆記本使用 What-If Tool

What-If Tool 是一種互動式視覺化介面，可協助您以圖表呈現資料集，並進一步瞭解機器學習模型的輸出內容。這是 Google PAIR 團隊建立的開放原始碼工具，雖然 Cloud Code 適用於任何類型的模型，但部分功能專為 Cloud AI Platform 而設計。

> **想進一步瞭解 What-If Tool 嗎？**歡迎前往網站和試用頁面，瞭解 PAIR 團隊，並觀看這部 YouTube 迷你系列影片。

在搭載 TensorFlow 的 Cloud AI Platform Notebooks 執行個體中，What-If Tool 已預先安裝。我們會使用這項指標，瞭解模型的整體效能，並檢查模型在測試集資料點上的行為。

步驟 1：準備 What-If Tool 的資料

為充分運用 What-If Tool，我們會從測試集傳送範例，以及這些範例的真值標籤 (`y_test`)。這樣一來，我們就能比較模型預測結果與真值。執行下列程式碼行，建立含有測試範例及其標籤的新 DataFrame：

```
wit_data = pd.concat([x_test, y_test], axis=1)
```

在本實驗室中，我們會將 What-If Tool 連線至剛在筆記本中訓練的模型。為此，我們需要編寫一個函式，供工具用來對模型執行這些測試資料點：

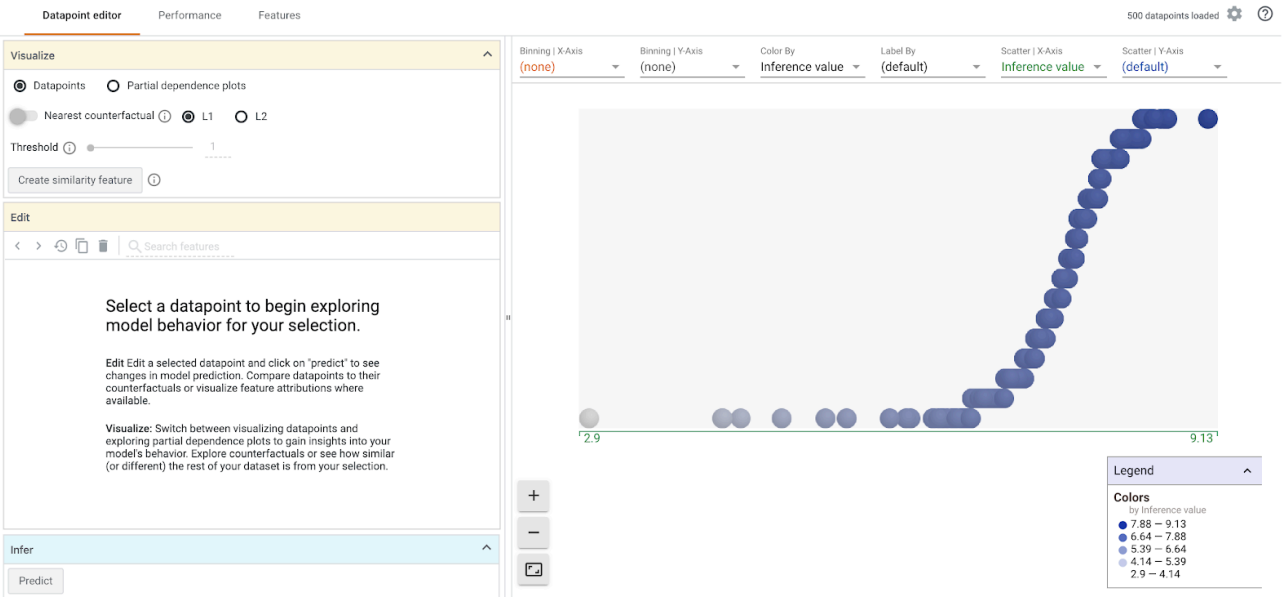
```
def custom_predict(examples_to_infer):
    preds = model.predict(examples_to_infer)
    return preds
```

步驟 2：例項化 What-If Tool

我們會傳遞剛才建立的串連測試資料集 + 真實標籤中的 500 個範例，來例項化 What-If Tool。我們建立 `WitConfigBuilder` 的執行個體來設定工具，並傳遞資料、上方定義的自訂預測函式、目標 (我們要預測的內容) 和模型類型：

```
config_builder = (WitConfigBuilder(wit_data[:500].values.tolist(), data.columns.tolist() +
['weight_pounds'])
  .set_custom_predict_fn(custom_predict)
  .set_target_feature('weight_pounds')
  .set_model_type('regression'))
WitWidget(config_builder, height=800)
```

載入 What-If Tool 後，畫面應如下所示：

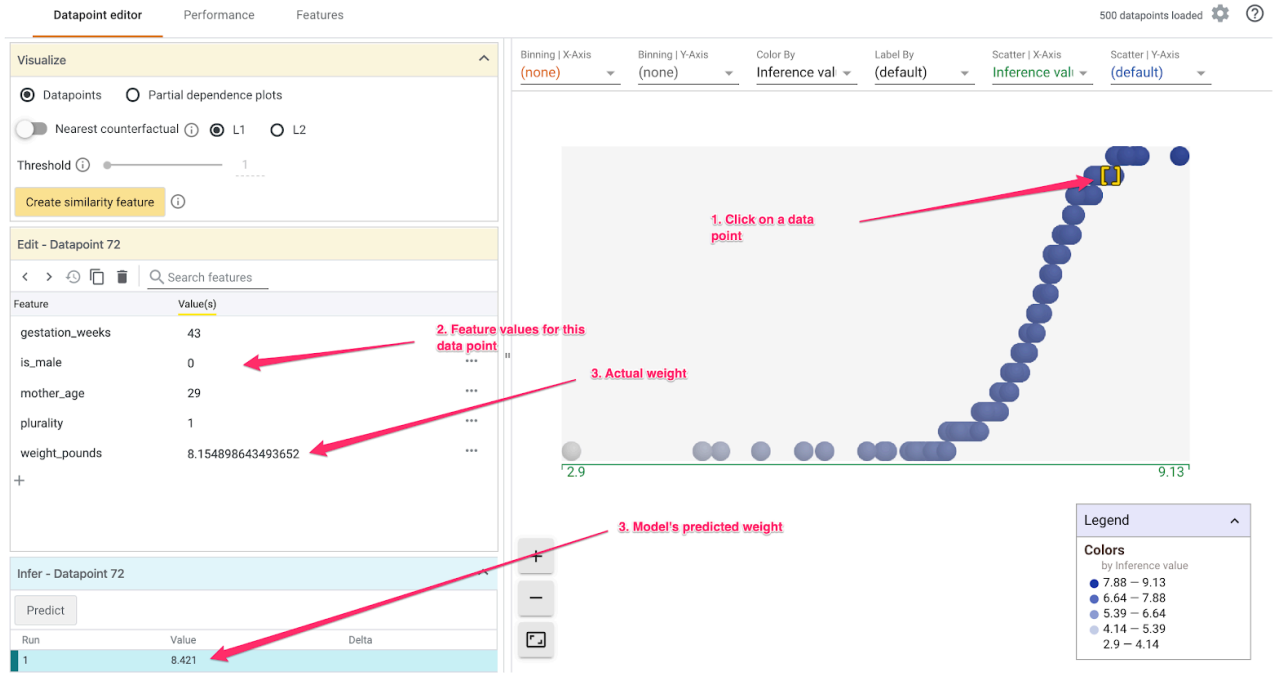


在 x 軸上，您可以看到測試資料點依模型預測權重值 `weight_pounds` 分散。

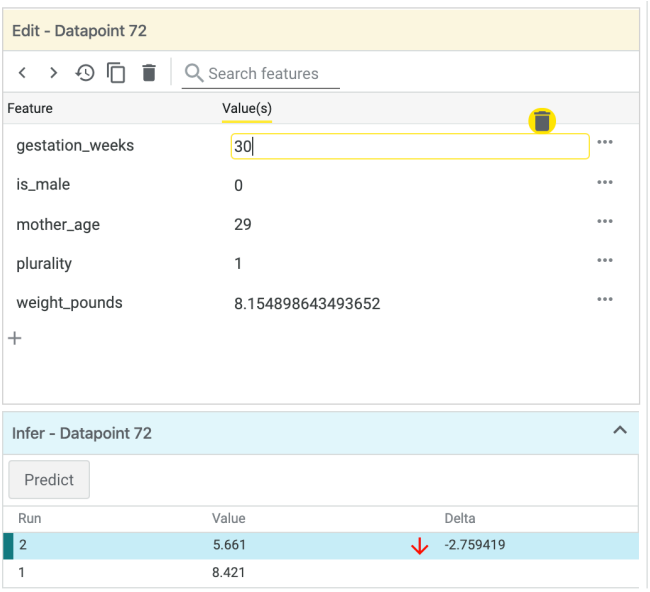
> 您也可以使用 `set_ai_platform_model` 方法，將 What-If Tool 直接連線至部署在 Cloud AI Platform 的模型。如需範例，請參閱這個試用版。

步驟 3：使用 What-If Tool 探索模型行為

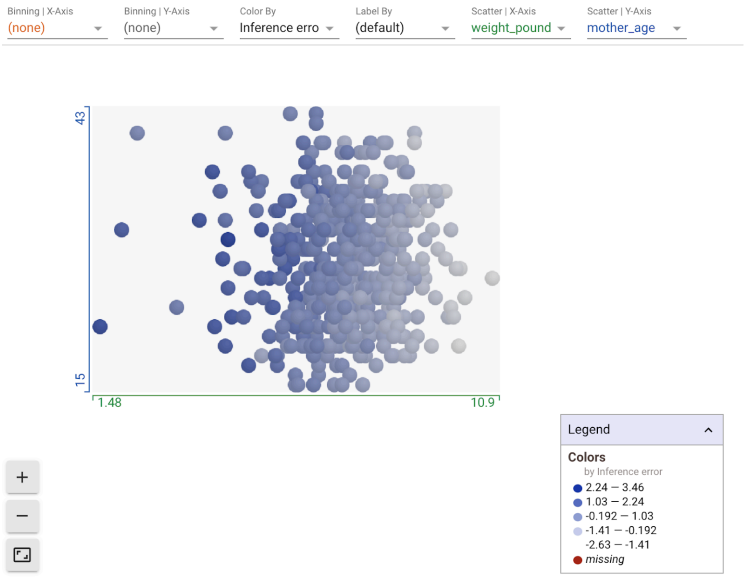
您可以使用 What-If Tool 執行許多有趣的操作。我們將在此探討其中幾項。首先，請查看資料點編輯器。您可以選取任何資料點來查看特徵，並變更特徵值。首先，按一下任一資料點：



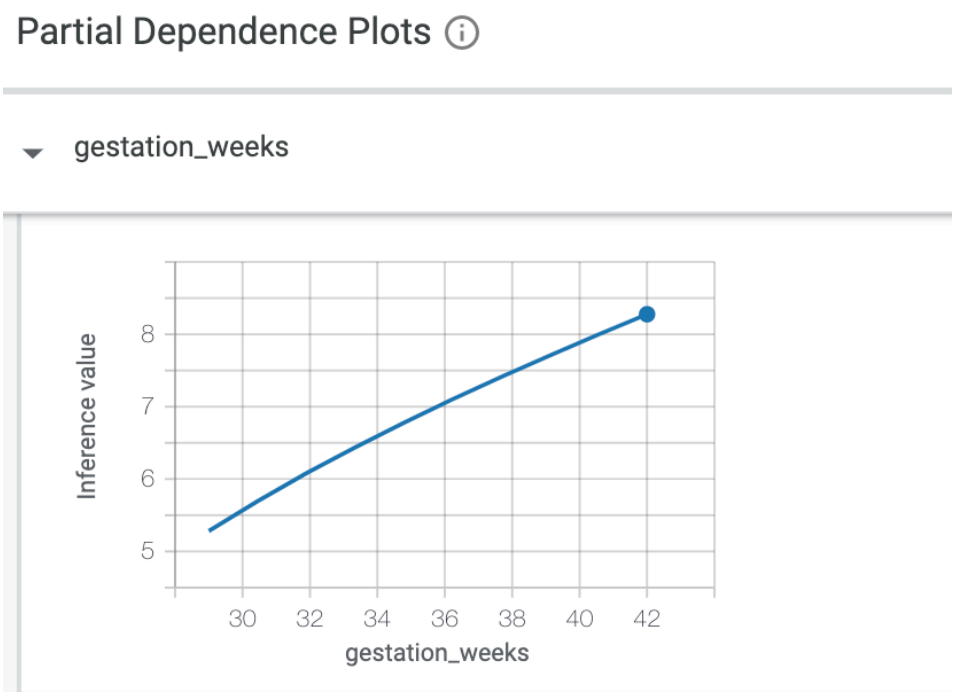
左側會顯示所選資料點的特徵值。您也可以比較該資料點的真值標籤與模型預測的值。您也可以在左側邊欄變更特徵值，然後重新執行模型預測，瞭解這項變更對模型的影響。舉例來說，我們可以雙擊這個資料點，將 gestation_weeks 變更為 30，然後重新執行預測：



您可以使用 What-If Tool 的繪圖部分中的下拉式選單，建立各種自訂視覺化效果。舉例來說，以下圖表顯示模型預測的體重 (X 軸)、母親年齡 (Y 軸)，以及依推論誤差著色的點 (顏色越深表示預測體重與實際體重的差異越大)。從下圖可看出，隨著權重減少，模型的錯誤率會稍微增加：



接著，勾選左側的「Partial dependence plots」(偏依賴圖) 按鈕。這項指標會顯示各項特徵對模型預測結果的影響。舉例來說，隨著妊娠時間增加，模型預測的嬰兒體重也會增加：



如要進一步瞭解如何使用 What-If Tool 進行探索，請參閱本節開頭的連結。

步驟 7 · 約 10 分鐘

7. 選用：將本機 Git 存放區連結至 GitHub

最後，我們會瞭解如何將筆記本執行個體中的 Git 存放區，連結至 GitHub 帳戶中的存放區。如要執行這個步驟，請先建立 GitHub 帳戶。

步驟 1：在 GitHub 上建立新的存放區

在 GitHub 帳戶中建立新的存放區。為存放區命名並加上說明，決定是否要公開，然後選取「建立存放區」 (不需要使用 README 初始化)。在下一個頁面中，按照指令列的指示推送現有存放區。

開啟終端機視窗，然後將新存放區新增為遠端存放區。在下列存放區網址中，將 `username` 換成您的 GitHub 使用者名稱，並將 `your-repo` 換成您剛建立的名稱：

```
git remote add origin git@github.com:username/your-repo.git
```

步驟 2：在筆記本執行個體中向 GitHub 進行驗證

接下來，您需要在筆記本執行個體中向 GitHub 進行驗證。這個程序會因您是否在 GitHub 上啟用雙重驗證而異。

如果您不確定從何著手，請按照 GitHub 文件中的步驟**建立 SSH 金鑰**，然後**將新金鑰新增至 GitHub**。

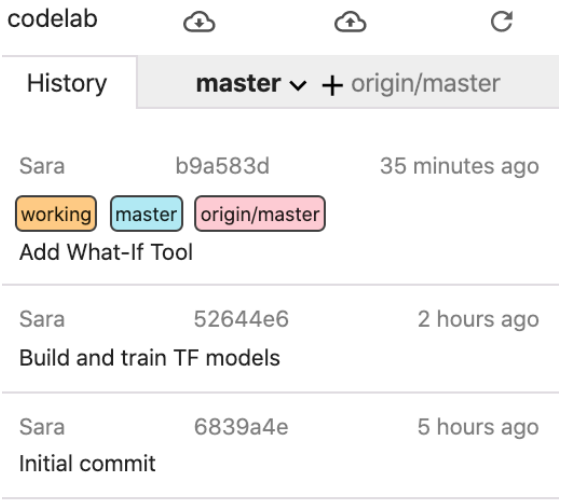
步驟 3：確認已正確連結 GitHub 存放區

如要確認設定正確無誤，請在終端機中執行 `git remote -v`。您應該會看到新存放區列為遠端項目。看到 GitHub 存放區的網址，並從筆記本向 GitHub 驗證後，即可直接從筆記本執行個體推送至 GitHub。

如要將本機筆記本 Git 存放區與新建立的 GitHub 存放區同步，請按一下 Git 側欄頂端的雲端上傳按鈕：

重新整理 GitHub 存放區，您應該會看到筆記本程式碼和先前的提交內容！如果其他人可以存取你的 GitHub 存放區，且你想將最新變更提取至筆記本，請按一下雲端下載圖示，同步處理這些變更。

在 Notebooks Git 使用者介面的「History」分頁中，您可以查看本機提交是否已與 GitHub 同步。在本例中，`origin/master` 對應 GitHub 上的存放區：



每當您進行新的修訂時，只要再次按一下雲端上傳按鈕，即可將這些變更推送至 GitHub 存放區。

步驟 8 · 約 1 分鐘

8. 恭喜！

您在本實驗室中學到許多內容 👏👏👏

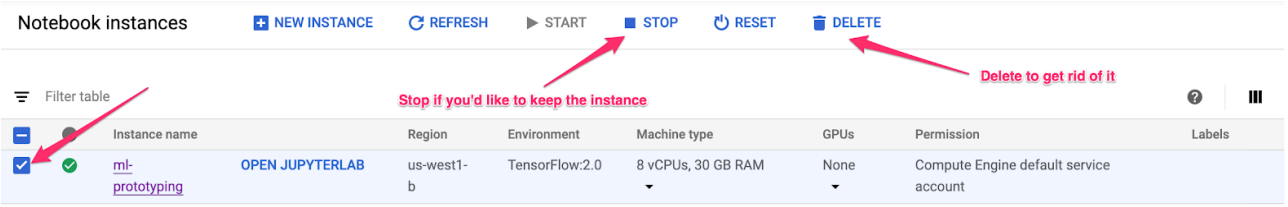
回顧一下，您已學會如何：

- 建立及自訂 AI Platform Notebook 執行個體
- 在該執行個體中初始化本機 git 存放區，透過 git UI 或指令列新增提交，在 Notebook git UI 中查看 git 差異
- 建構及訓練簡單的 TensorFlow 2 模型
- 在 Notebook 執行個體中使用 What-If Tool
- 將 Notebook Git 存放區連結至 GitHub 上的外部存放區

步驟 9 · 約 1 分鐘

9. 清除所用資源

如要繼續使用這部筆電，建議在不使用時關機。在 Cloud 控制台的 Notebooks 使用者介面中，選取筆記本，然後選取「停止」：



如要刪除在本實驗室中建立的所有資源，請直接刪除筆記本執行個體，而不是停止執行個體。

使用 Cloud 控制台的導覽選單瀏覽至「儲存空間」，然後刪除您建立的兩個 bucket，以儲存模型資產。